

PatchIQ-iA Security Analysis

Sample 03 — Command Injection Detection

1. Objective

The objective of this test is to evaluate whether PatchIQ-iA can detect a command injection vulnerability when user-controlled input is passed directly to the `os.system()` function. The test uses a small command-line utility to represent a realistic application scenario.

2. Input Code

The following Python code was submitted as part of the Pull Request. The code contains an intentionally vulnerable use of `os.system()`.

```
import os

def run_command():
    print("System Command Tool")

    command = input("Enter command: ")

    if not command:
        print("No command entered.")
        return

    print("Running:", command)

    # Vulnerable: user input is passed directly to os.system()
    os.system(command)

def main():
    print("PatchIQ-iA Command Utility")
    print("-----")

    choice = input("Run a system command? (yes/no): ")

    if choice.lower() == "yes":
        run_command()
    else:
        print("Operation cancelled.")

if __name__ == "__main__":
    main()
```

3. PatchIQ-iA Output

The following screenshot shows the actual security review generated by PatchIQ-iA for this Pull Request.

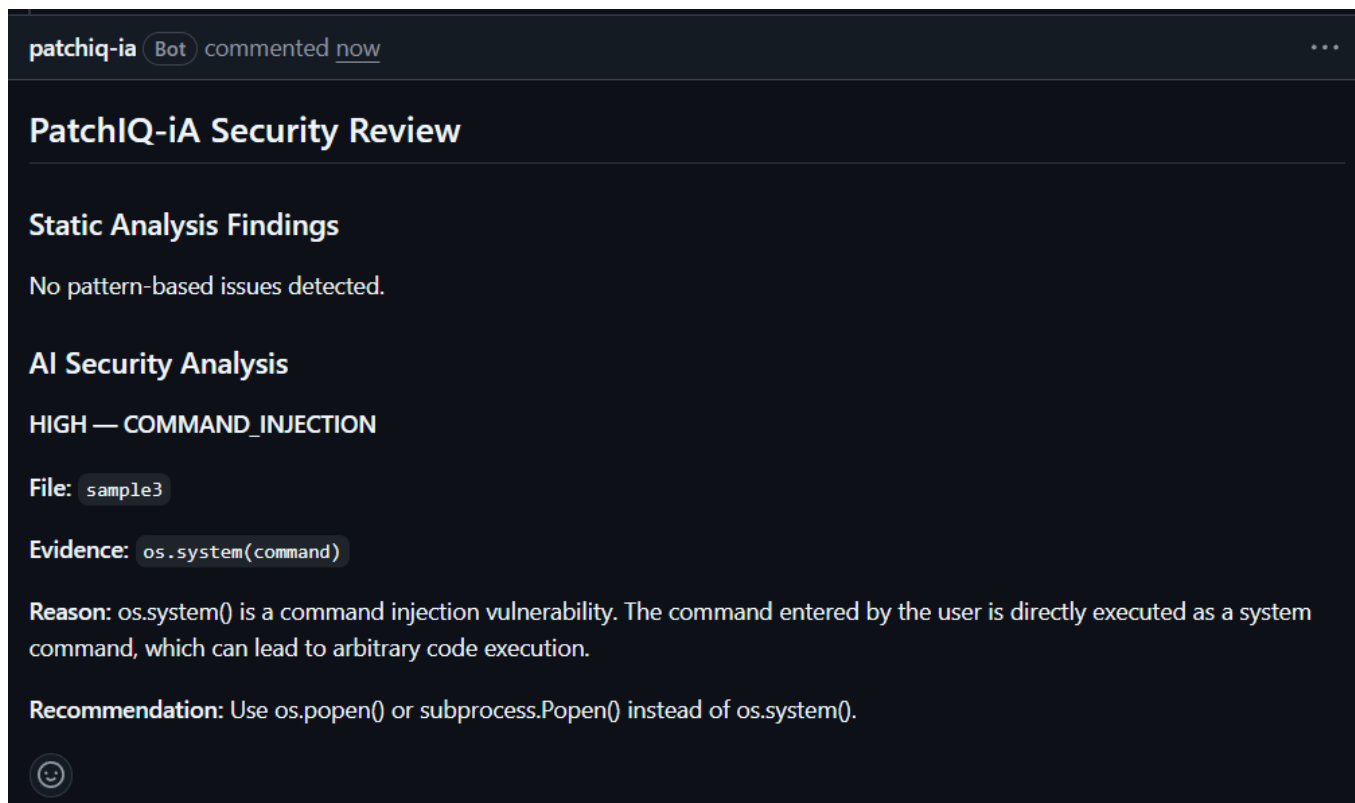


Figure 1. Actual PatchIQ-iA Security Review generated during the test.

4. Conclusion

PatchIQ-iA successfully detected the command injection vulnerability through AI security analysis. The finding was classified as HIGH severity, with evidence identifying the vulnerable `os.system(command)` call. The analysis also provided reasoning explaining that user-controlled input is directly executed as a system command and recommended using safer process execution mechanisms instead of `os.system()`.